--- status: verified verified: 2026-09-18 verified_by: headless session 822ae304, Windows 11 Pro 10.0.26200, Node v24.15.0 expires_when: > The 32 KB always-loaded rules budget changes, the harness starts rendering every skill description, `higgsfield-ai/skills` publishes an automated eval runner, or a re-run of the transcript census puts model-initiated `Skill` calls outside 100-170 per 7 days. ---

Higgsfield's skill and workflow architecture, against our measured gap

Higgsfield ships two things that look like a harness: a public Claude Code skill marketplace (`higgsfield-ai/skills`, MIT, never evaluated here) and an MCP server whose workflows load in three stages, with the trigger in the always-loaded stage. **We should care about the mechanism and adopt none of the code**: every mechanical idea in it is already in this repo, and the one thing that is not, an always-loaded router, loses on arithmetic against a 32 KB budget with 60 bytes of headroom. The surprise is that the strongest evidence for their design is already running here and is not theirs: `artifact-design` fired **67** times in 7 days, more than the other 32 skills combined, because an always-loaded **tool description** orders the model to load it, while `dataviz`, whose own description carries a louder trigger list and no such order, fired **2**.

---

1. What is it for, and who decided that?

Three different artifacts carry the Higgsfield name. Conflating them is how the previous study got a stale question.

artifact	what it is	licence	read at
`wide-trace/open-higgsfield`	a Next.js media-generation studio, third-party	**none**	`b16a0efe4`
`higgsfield-ai/skills`	a Claude Code / Codex / Cursor plugin marketplace, 8 skills	**MIT**	`d0714061`
`mcp__higgsfield__*`	the vendor's MCP server, connected to this session	n/a (hosted)	live, 2026-09-18

The prior verdict is unchanged, because the repo is unchanged

`[stated 2026-09-18]` Andy: the Higgsfield harness "is now open source in full". That reopened `open-higgsfield-evaluated-2026-09-18`, whose rejection rested on two premises. Both still hold, at the same commit the first study read:

```
gh api repos/wide-trace/open-higgsfield --jq '{license,pushed_at,stargazers_count}'
# {"license":null,"pushed_at":"2026-09-17T01:53:26Z","stargazers_count":3040}
gh api repos/wide-trace/open-higgsfield/license          # 404 Not Found
gh api repos/wide-trace/open-higgsfield/contents/ --jq '.[].name'   # no LICENSE
gh api repos/wide-trace/open-higgsfield/commits --jq '.[].sha[0:9]'
# b16a0efe4  eac4a2563  e123319ba   (three commits, total)
```

`[measured 2026-09-18]` `license: null`, no `LICENSE` file in the root tree, the licence endpoint 404s, and HEAD is **`b16a0efe4`**, the same commit as the earlier evaluation. Zero commits have landed since. The 25 rejections were not re-derived, because the tree they were about did not move. **That repo is not what Andy is pointing at.**

What he is pointing at, most likely

`higgsfield-ai/skills` is the vendor's own org, MIT-licensed, and it *is* harness-shaped: a `.claude-plugin/marketplace.json`, eight skills with `references/` for on-demand loading, an `evals/` directory and a CI workflow that grades skill frontmatter. It was never evaluated here.

```
gh api repos/higgsfield-ai/skills --jq '{license:.license.spdx_id,stargazers_count,forks_count}'
# {"license":"MIT","stargazers_count":1040,"forks_count":189}
```

Its purpose, in one sentence: **route a user's natural-language media request to exactly one of eight skills, each of which shells out to one CLI binary.** Audience: a person in Claude Code who wants an image, a video, a brandbook or a website, and who never learns the model names.

2. What is actually there? Name the population.

```
git clone https://github.com/higgsfield-ai/skills.git hf-skills
git -C hf-skills log -1 --format='%H %ad' --date=iso   # d071406147a37b835bed09543d85ab3e9bd85c7d
2026-09-11
git -C hf-skills ls-files | wc -l                      # 129
git -C hf-skills log --oneline | wc -l                 # 89
git -C hf-skills log --since="90 days ago" --format='%ae' | sort | uniq -c | sort -rn
#   49 nurdaulet@higgsfield.ai
#    8 arsu@higgsfield.ai
#    2 edige.akimali@nu.edu.kz
#    1 115921160+Akim-Edige@users.noreply.github.com
```

`[measured 2026-09-18]` **Alive**: 60 of 89 commits in the last 90 days, four distinct authors, three of them on a company address. This is maintained work, not an archive.

The skill bodies are large and the references larger: `higgsfield-generate/SKILL.md` is 25,012 bytes, `higgsfield-websites/SKILL.md` 14,951, and `higgsfield-websites/references/design-taste-frontend.md`

87,674. **The heavy content is on disk and loads on demand**; only the frontmatter `description` is meant to be resident.

The MCP server's catalog is a separate population:

```
mcp__higgsfield__get_workflow_instructions   (no argument)
→ {"mode":"catalog","source":"bundled_resource","workflows":[ …17 entries… ]}
```

`[measured 2026-09-18]` **17 workflows**, each carrying `name`, `version` and a description averaging roughly 570 characters. The catalog carries no instructions at all.

3. How does it actually run? The three stages.

This is the design worth understanding, and it is three stages, not two.

stage	what it is	when it is paid	approx size
1	the MCP server's `instructions` block, injected into the system prompt	**every session, unconditionally**	~2.6 KB observed, and **truncated** by the harness
2	`get_workflow_instructions` with no argument: the catalog	only when stage 1 fires	~9.8 KB, 17 descriptions
3	`get_workflow_instructions {workflow}` and `get_workflow_bundle_file`	only when stage 2 routes	~16 KB for `subtitles` alone

Stage 1 is the whole trick, and its content is not documentation. It is **imperative, conditional and negative**, all three:

> "Before any multi-step made-to-brief video — narrated explainer or story, ad / commercial, UGC / talking head, podcast, or similar — call `get_workflow_instructions` first (with no argument), then load the matching workflow. **Always check the catalog rather than assuming which workflows exist.**" "**Never use legacy `motion_control` or `ad-multiplier` for a single Genjutsu edit.**"

Stage 2's entries repeat the shape at a finer grain. Three properties, present in all 17:

1. **Literal quoted user phrases.** `ugc-unboxing-video` triggers on "haul", "PR drop", "opening the box". Not a category, a string the user will actually type.
2. **A conjunctive gate.** "Triggers only with ALL of: (a) unboxing / reveal intent, (b) UGC framing, (c) a specific product (photo, URL, or 'this/our/my X')."
3. **A negative boundary that names the alternative.** "NOT for: a talking-head review with no unboxing arc (`ugc-review-video`), product-only with no creator (`ugc-product-video`)…" Every one of the six `ugc-*` workflows names the other five. The descriptions are not eight summaries, they are **one routing graph** distributed across eight files.

One structural property that matters and is easy to miss: **stage 2's text is the stage-3 file's own YAML frontmatter.** The catalog is generated from the SKILL.md that the catalog routes to, so the trigger cannot drift from the body it triggers. That is the same rule this repo applies to `gate-fast.js`, which derives its step list from `scripts.gate` rather than copying it.

What proves a change is good, on their side

```
# .github/workflows/validate-skills.yml — the frontmatter job, in full substance
if fm.get("name") != expected_name:      errors.append(...)
if not fm.get("version"):                errors.append("version missing")
if len(desc) > 1024:                     errors.append("description exceeds 1024 characters")
if "Use when" not in desc:               errors.append("description missing 'Use when' trigger
phrases")
if "NOT for" not in desc:                errors.append("description missing 'NOT for' boundary")
```

`[measured 2026-09-18]` **A PR fails if a skill description omits the literal string `NOT for`.** The routing graph is not a convention, it is a gate. Runtime: a Python frontmatter parse over 8 files, so seconds. What it does **not** cover: whether the description is any good, whether the skill works, or whether it ever fires. Their `evals/README.md` says so plainly: "*There is no automated runner yet.*"

4. Where is the risk, and does the pattern actually fire?

This is the load-bearing measurement, and the probe needed a control, because a near-zero count can mean the mechanism fails or can mean the condition never arose.

```
node count-tools.js 7   # every .jsonl under ~/.claude/projects modified in 7 days
# transcripts=613 lines=686773 parsed=686773 tool_use=64576
```

`[measured 2026-09-18]` **613 transcripts, 686,773 JSONL lines, all parsed, 64,576 `tool_use` blocks.** Population stated so the zeros below are readable.

tool	calls, 7d
`Bash`	34,381
`Edit`	6,691
`Read`	4,931
`ToolSearch`	1,098
`Skill`	**136**
`mcp__higgsfield__*` (all)	**13**, in 5 transcripts
`mcp__higgsfield__get_workflow_instructions`	**2**

Two firings is not a verdict on its own. The sequences are:

```
71d431f6  get_workflow_instructions{website-builder-flow} -> list_websites -> balance
          -> get_workflow_bundle_file{website-builder-flow} -> models_explore x2
          -> get_workflow_bundle_file{website-builder-flow}
c6c59c9e  balance -> models_explore -> models_explore
0db4331a  balance
a97dbec7  balance
822ae304  get_workflow_instructions          (this session, and I was told to call it)
```

`[measured 2026-09-18]` **The pattern fired correctly and unprompted exactly once** — session `71d431f6` routed through the catalog *before* touching a website tool, then pulled bundle files twice. In the three sessions that only read balance and models, the trigger correctly did **not** fire; those are not multi-step video briefs. So the control holds in both directions: **1 of 1 qualifying occasions fired, and 0 of 3 non-qualifying occasions fired spuriously.** n=1 is n=1, and the honest reading is that the trigger condition almost never arises on this machine, not that the mechanism is weak.

The natural experiment that is already running here

Our side of the same question, from the same census:

```
node skill-origin.js
# Skill calls 7d: user-typed=6 model-initiated=130
```

`[measured 2026-09-18]` **136 `Skill` calls, 33 distinct skills, 130 of 136 model-initiated.** The brief's "6 of 53 skills fired" is stale for the whole library; it still holds for autodev-core alone, which contributed **9 distinct skills of 59**.

The distribution is the finding:

skill	calls, 7d	is it named in an always-loaded imperative?
`artifact-design`	**67**	yes — the `Artifact` tool description: "*Before writing the file, Claude must load the `artifact-design` skill*"
`artifact-capabilities`	**10**	yes — same tool description, "*Claude must load the `artifact-capabilities` skill before writing the artifact*"
`vercel-kb`, `claude-api`	5 each	no
`dataviz`	**2**	no — its own description carries a long explicit trigger list
28 others	1-4 each	no

77 of 130 model-initiated skill loads (59.2%) come from the two skills that an always-loaded tool description orders the model to load. They are 2 of roughly 459 installed skills.

The compliance rate, which is the number that makes it causal rather than coincidental:

```
node artifact-vs-skill.js
# Artifact publish calls=166 in 74 sessions; other Artifact actions=113
# artifact-design loads=67 in 63 sessions
# sessions that published AND loaded artifact-design=61
# sessions that published but NEVER loaded artifact-design=13
```

`[measured 2026-09-18]` **61 of the 74 sessions that published an artifact loaded `artifact-design` (82.4%).** `[inferred]` the imperative is what produces that, resting on the `dataviz` contrast: a comparably specific trigger living in the skill's own description, with no tool-side order, yielded 2 loads. This is an observational contrast, not an ablation — I did not remove the mandate and re-measure, and could not.

A third instance of the same mechanism, unrelated to artifacts: **`ToolSearch` fired 1,098 times.** It is pure two-stage loading (names visible, schemas fetched on demand) and its stage-1 half is an imperative in a system-reminder: "*Use ToolSearch with query `select:<name>` to load tool schemas before calling them.*" Three instances, three mechanisms, one shape.

Where the risk is, for us

- **The always-loaded budget is full.** `[measured 2026-09-18]` the 15 `rules/*.md` files with no `paths:` frontmatter total **28,472 bytes**, plus `CLAUDE.md` at **3,468** = **31,940 of 32,000**. Sixty bytes. Any router paid out of this budget has to delete a rule to fund itself.
- **Stage 1 can be truncated.** The higgsfield MCP `instructions` block in this session's system prompt ends mid-sentence with `[truncated]`. An always-loaded trigger is only as reliable as the harness's willingness to render all of it — the same failure that drops 421 of 459 skill descriptions.
- **A memory names a tool that no longer exists.** `skill-descriptions-mostly-not-rendered` says to run `node tooling/skill-census.js --rendered`; that file is not in `tooling/`. Corrected in the memory this session.

5. What would I tell someone starting tomorrow?

Judge each idea against a gap we measured. Four ideas, one partial adoption, no code.

#	idea	verdict	why, with the number
I1	Two-stage load with the trigger in the always-loaded half	**REJECT as a rules-budget router; ALREADY OURS as a mechanism**	We have three working instances (`ToolSearch` 1,098, `artifact-design` 67, `artifact-capabilities` 10). In every one, stage 1 is a **tool description or system-reminder**, which we do not author and which is off the 32 KB budget. Higgsfield gets the same slot via MCP `instructions`. We have 60 bytes of headroom and no MCP server of our own.
I2	CI gate on description shape (`Use when` + `NOT for` + ≤1024 chars)	**ALREADY OURS, and clean**	`tooling/check-skill-triggers.js` exists and its header states the same thesis. It reports `names no condition: 0 of 68`, `overlong: 0`, `no when_to_use: 0 of 68`. Porting their gate would find zero defects.
I3	The negative boundary that names the alternative	**PARTIAL — the one real gap**	`[measured]` **3 of our 68 plugin descriptions carry a negative boundary; 8 of 8 of theirs do.** But `[measured 2026-09-18, prior]` only ~38 of ~459 descriptions get a rendered slot, so editing the other 421 is unmeasurable. Add it when a rendered skill is being edited anyway. Not a campaign.
I4	`evals/` as scored scenarios with a round protocol	**ALREADY OURS, and ours is better**	Theirs is 10 markdown scenarios scored by a human; their README says "*There is no automated runner yet.*" We have an automated eval suite, which is how we know the delta is +0.02.

The arithmetic that kills the router, in full

Sessions per day: **613 transcripts / 7 days = 87.6** (transcripts, not sessions; a session can span files, so this is an upper bound on sessions and therefore conservative against the router).

A 3 KB always-loaded router is ~750 tokens, resident in every session: **750 x 87.6 = ~65,700 tokens/day of standing cost**, paid whether any skill loads or not, and it cannot be paid at all without deleting ~3 KB of rules.

Against it: skills fire **136 times per 7 days = 19.4/day**, and the eval suite put the quality delta of a skill firing at **+0.02**. Even a router that **doubled** firing buys 19.4 additional firings x 0.02 = **+0.39 quality units per day, for 65,700 tokens per day**. The measured value of the thing being bought does not cover the standing cost by any margin, and the budget cannot fund it regardless.

The conditional alternative survives the arithmetic where the router does not: autodev-core already registers `UserPromptSubmit` hooks (`user-prompt-image-scan.js`, `inbox-notify.js`), and a hook that injects a one-line imperative only on a matching prompt costs **zero bytes on non-matching prompts and zero bytes of the 32 KB budget**. That is the shape any future fix should take. **It was not built**, because the thing it would buy is worth +0.02 per firing and the brief's own evidence points the other way: the three EB redesign sessions produced the best work we have shipped, across 3,476 tool calls, with `Skill` firing 3 times. **More skill firing is not an established good, so spending context to cause it is not yet justified.**

The three things that would cost someone a day

1. **`open-higgsfield` and `higgsfield-ai/skills` are different projects with different licences.** One is unlicensed and unchanged since the last study; the other is MIT and maintained. Check the org before deciding anything.
2. **Rewriting a skill's description is unmeasurable for 421 of 459 skills**, because the harness renders only ~38 description slots. Check whether the skill holds a slot before editing its description as a fix.
3. **The lever on whether a skill fires is not the skill.** It is whether something always loaded orders the model to load it. `artifact-design` is not a better-written skill than `dataviz`; it is a skill named in a tool description, and that is worth 67 loads against 2.

What could not be established

- **No ablation.** The causal claim about the mandate rests on an observational contrast (`dataviz`) and a compliance rate (82.4%). Removing the mandate and re-measuring is not something this session could do.
- **n=1 for the Higgsfield pattern firing unprompted.** One qualifying occasion, one correct firing. That is a positive existence proof and not a rate.
- **Stage-1 byte size is approximate.** The MCP `instructions` block was read from the system prompt, where it is truncated, so ~2.6 KB is a floor, not a measurement.
- **The "42 descriptions name a sibling skill" probe is not trustworthy** and is excluded from the conclusions: it substring-matches short skill names like `test`, `core`, `auto` and `design`, which appear in unrelated prose. The negative-boundary count (3 of
- uses phrase markers and is the number I stand behind.
- **`higgsfield-ai/cli`, the binary every skill shells out to, has no source in its repo** (LICENSE, README, MODELS.md, `install.sh` only), so the skills were judged on their instructions, not on what the CLI does.

What would change this

- **The 32 KB always-loaded budget grows, or a rule is retired to make room.** The router rejection is arithmetic over that constant; change the constant and re-run it.
- **The harness starts rendering every skill description.** Then I3 stops being unmeasurable and the negative-boundary edit becomes worth a campaign.
- **The eval suite measures a skill-firing delta materially above +0.02.** The whole cost/benefit above is one multiplication against that number.
- **We ship an MCP server of our own.** That would hand us the same always-loaded `instructions` slot Higgsfield uses, off the rules budget, and I1 would need re-deciding on the spot.

- **`higgsfield-ai/skills` ships the automated eval runner its README promises.** I4 is rejected only because theirs is manual and ours is not.

Appendix: every command

```
# licence and liveness, third-party repo
gh api repos/wide-trace/open-higgsfield --jq '{license,pushed_at,stargazers_count}'
gh api repos/wide-trace/open-higgsfield/license
gh api repos/wide-trace/open-higgsfield/contents/ --jq '.[].name'
gh api repos/wide-trace/open-higgsfield/commits --jq '.[].sha[0:9]'

# the vendor org
gh api "search/repositories?q=higgsfield&sort=stars&per_page=15" \
  --jq '.items[] | "\(.stargazers_count)\t\(.full_name)\t\(.license.spdx_id // "NONE")"'
gh api orgs/higgsfield-ai/repos --jq '.[] | "\(.name)\t\(.license.spdx_id // "NONE")"'

# population of higgsfield-ai/skills
git clone https://github.com/higgsfield-ai/skills.git hf-skills
git -C hf-skills log -1 --format='%H %ad' --date=iso
git -C hf-skills ls-files | wc -l
git -C hf-skills log --oneline | wc -l
git -C hf-skills log --since="90 days ago" --format='%ae' | sort | uniq -c | sort -rn
awk '/^---$/{n++; next} n==1{print} n==2{exit}' hf-skills/higgsfield-*/SKILL.md

# the MCP three stages (tool calls, not shell)
#   mcp__higgsfield__get_workflow_instructions {}
#   mcp__higgsfield__get_workflow_instructions { workflow: "subtitles" }

# our side — one-off probes, written to a scratch dir and not committed as tooling
# (every tooling/ script is probed by check:entrypoints, and these are not entry points).
# The census walks ~/.claude/projects/**/*.jsonl with mtime within N days. Reproduce it:
```

Save as `census.js` and run `node census.js 7`:

```
const fs = require('fs'), path = require('path');
const root = process.env.USERPROFILE + '/.claude/projects';
const cutoff = Date.now() - Number(process.argv[2] || 7) * 864e5;
const files = [];
(function walk(d) { for (const e of fs.readdirSync(d, { withFileTypes: true })) {
  const p = path.join(d, e.name);
  if (e.isDirectory()) walk(p);
  else if (e.name.endsWith('.jsonl')) { try { if (fs.statSync(p).mtimeMs >= cutoff) files.push(p); }
catch {} }
} })(root);
const counts = new Map(); let typed = 0, model = 0, lines = 0, toolUses = 0;
for (const f of files) { let txt; try { txt = fs.readFileSync(f, 'utf8'); } catch { continue; }
  let lastUser = '';
  for (const line of txt.split('\n')) { if (!line.trim()) continue; lines++;
    let o; try { o = JSON.parse(line); } catch { continue; }
    const m = o && o.message; if (!m) continue;
    if (m.role === 'user') { const c = m.content;
      lastUser = typeof c === 'string' ? c
        : (Array.isArray(c) ? c.filter(b => b && b.type === 'text').map(b => b.text).join('\n') :
''); }
    if (m.role !== 'assistant' || !Array.isArray(m.content)) continue;
    for (const b of m.content) { if (!b || b.type !== 'tool_use') continue; toolUses++;
      counts.set(b.name, (counts.get(b.name) || 0) + 1);
      if (b.name === 'Skill') { const bare = String(b.input && b.input.skill || '').split(':').pop();
        if (lastUser.indexOf('/' + bare) !== -1) typed++; else model++; } } } }
console.log(`transcripts=${files.length} lines=${lines} tool_use=${toolUses}`);
console.log(`Skill: user-typed=${typed} model-initiated=${model}`);
for (const e of [...counts].sort((a, b) => b[1] - a[1]).slice(0, 12))
console.log(String(e[1]).padStart(7), e[0]);
```

The other four probes are the same walk with a different tally: higgsfield call sequences per transcript, `Skill` by `input.skill`, `Artifact` publishes against `artifact-design` loads per session, and a frontmatter scan of `plugins/**/SKILL.md` for the phrase markers `not for`, `instead of`, `rather than`, `do not use`.

```
# our always-loaded budget
cd ~/.claude/rules && tot=0; for f in *.md; do \
  head -12 "$f" | grep -q '^paths:' || { s=$(wc -c < "$f"); tot=$((tot+s)); }; done; echo $tot
wc -c < ~/.claude/CLAUDE.md

# our existing equivalent of their CI gate
cd ~/claude-auto-dev && node tooling/check-skill-triggers.js
```